Lecture 5: LLMs for Business Valuation

Large Language Models in Finance

Juan F. Imbet

Paris Dauphine – PSL University

Outline

What Is Business Valuation?

Definition

Business valuation maps observable and projected financial variables — cash flows, discount rates, growth rates — into a scalar enterprise value $V \in \mathbb{R}_+$ that a rational, fully informed market participant would assign to the claim.

Professional contexts:

- **M&A**: spread between buyer and seller estimates defines zone of possible agreement
- **IPOs**: investment banks produce valuation ranges to price new shares
- **Portfolio management**: model value vs. market price generates trade signals
- **Financial reporting**: IFRS 13 / ASC 820 Level 3 assets require documented models

What Is Business Valuation? (cont.)

THE BIGGER PICTURE

Valuation is forward-looking. A balance sheet records historical cost; the market prices a claim on *future* earnings. Translating current information into a credible forward estimate is where LLMs add the most value.

Three Valuation Frameworks

Income approach — DCF:

$$V = \sum_{t=1}^{n} \frac{\text{FCF}_t}{(1+\text{WACC})^t} + \underbrace{\frac{\text{FCF}_{n+1}/(\text{WACC}-g)}{(1+\text{WACC})^n}}_{\text{Terminal Value}}$$

A 100-bps change in WACC or g alters terminal value by **20–40%**. Scenario analysis is indispensable.

Market approach — trading multiples (EV/EBITDA, P/E, P/Book) from comparable firms. Challenge: finding genuinely comparable peers. *LLMs and embeddings help here.*

Asset approach — net asset value:

$$V_{\text{NAV}} = \sum_i FV(\text{Asset}_i) - \sum_j FV(\text{Liability}_j)$$

Best for asset-intensive firms or as a floor valuation cross-check.

In practice

Triangulate across all three: present a valuation *range*, not a point esti-

How LLMs Transform Each Valuation Step

Step	Traditional approach	LLM contribution
Data collection	Manual EDGAR parsing	Automated extraction pip
FCF computation	Spreadsheet formulas	Code interpreter (error-fre
Forecasting	Analyst judgment	CoT prompting + evaluat
Scenario analysis	Manual scenarios	Narrative coherence check
Comparable selection	SIC codes + manual	Embedding + LLM hybric
Report generation	Analyst writing	LLM first-draft synthesis

THE BIGGER PICTURE

Key principle: LLMs are most valuable where the task is *qualitative reasoning over text* and least reliable where the task is *numerical computation*. Route arithmetic to a code interpreter.

The SEC EDGAR Ecosystem

Key filing types for valuation:

- **10-K**: annual audited financials + MD&A. Primary source.
- **10-Q**: quarterly unaudited interim statements.
- **8-K**: material events within 4 business days (earnings, M&A, CEO changes).
- **DEF 14A**: proxy statement — executive comp, board composition.

Programmatic access:

- Submissions API: `data.sec.gov/submissions/CIK{cik}.json`
- Full-text search: `efts.sec.gov/LATEST/search-index?q=...`
- XBRL companyfacts: `data.sec.gov/api/xbrl/companyfacts/CIK{cik}.json`

XBRL: the first-pass extractor

Since 2009, the SEC requires XBRL tagging. XBRL covers the majority of standard line items reliably. LLM parsing fills the gap for non-standard layouts, merged cells, and extension elements.

Parsing Financial Tables with LLMs

Why rule-based approaches fail: merged cells, multi-level headers, footnote references inside cells, non-standard layouts. Classical parsers break on the long tail.

UNDER THE HOOD

LLM extraction pattern:

1. Extract raw HTML of candidate table with BeautifulSoup
2. Convert to compact plain-text (preserve headers, strip HTML tags)
3. Send to LLM with structured extraction prompt requesting JSON schema
4. Validate output against schema; retry on parse failure

Parsing Financial Tables with LLMs (cont.)

Design principles:

- Request JSON directly — reduces post-processing; enables automatic validation
- Add "Respond only with valid JSON. No markdown code fences." if model lacks JSON mode
- Use XBRL as ground truth; flag conflicts between XBRL tags and LLM extractions

Avoid look-ahead bias

Always record the *filing date*, not the fiscal year end date. Using fiscal year end in a backtest can include information not yet available.

Free Cash Flow: Two Canonical Definitions

UNDER THE HOOD

Free Cash Flow to Firm (FCFF):

$$\mathrm{FCFF}_t = \mathrm{EBIT}_t(1-\tau) + \mathrm{DA}_t - \mathrm{CapEx}_t - \Delta\mathrm{NWC}_t$$

Free Cash Flow to Equity (FCFE):

$$\mathrm{FCFE}_t = \mathrm{NI}_t + \mathrm{DA}_t - \mathrm{CapEx}_t - \Delta\mathrm{NWC}_t + \mathrm{NB}_t$$

Relationship (Modigliani–Miller with taxes):

$$\mathrm{FCFE}_t = \mathrm{FCFF}_t - \mathrm{InterestExp}_t(1-\tau) + \mathrm{NB}_t$$

Which to use? When leverage is changing materially, FCFF is more robust: WACC can be held constant while cost of equity must be re-levered per scenario. (See appendix for the discount-rate/output mapping.)

Worked Example: FCFF Computation

Hypothetical software company (USD millions):

Revenue	4,200
EBIT	840
Depreciation & Amortisation	310
Capital Expenditure	180
NWC (current year)	620
NWC (prior year)	540
Tax rate τ	21%

$$\begin{aligned} \Delta\text{NWC} &= 620 - 540 = 80 \\ \text{FCFF} &= 840(1 - 0.21) + 310 - 180 - 80 \\ &= 663.6 + 310 - 180 - 80 = \mathbf{713.6}\ \text{M} \end{aligned}$$

With WACC = 9%, 5-year explicit growth = 8%, terminal growth = 3%: Enterprise value ≈ **$15.2B** ≈ 3.6× trailing revenue.

Normalising EBIT: LLM-Assisted Non-Recurring Items

Three categories that distort FCFF if treated naively:

1. **One-time items**: restructuring charges, litigation settlements, impairments. Remove from base EBIT used for forecasting.
2. **Capitalised operating leases** (IFRS 16, ASC 842): mismatch between reported EBIT, depreciation, and cash flows; must be reconciled.
3. **Acquired intangible amortisation**: from purchase accounting step-up; often excluded in "cash EBIT" calculations.

LLM-assisted normalisation: prompt the model on the MD&A to identify non-recurring items; return JSON with description, amount, and confidence (High / Medium / Low).

$$\mathrm{EBIT}^* = \mathrm{EBIT} + \sum_{k \in \mathcal{K}} a_k$$

Human-in-the-loop design

High confidence → auto-strip. Medium → flag for analyst review. Low → pass through. The LLM accelerates identification; the analyst makes

Classical Forecasting Baselines

Random walk with drift:

$$\mathsf{FCF}_{t+1} = \mu + \mathsf{FCF}_t + \varepsilon_{t+1}, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

Mean-reverting model:

$$\mathsf{FCF}_{t+1} - \bar{\mathsf{FCF}} = \phi(\mathsf{FCF}_t - \bar{\mathsf{FCF}}) + \varepsilon_{t+1}$$

Empirical estimates: $\phi \approx 0.5$–0.7 for earnings — substantial persistence, non-trivial mean reversion.

Key limitation of pure time-series models: They rely exclusively on historical values. They ignore qualitative information in management guidance, industry trends, and macroeconomic conditions.

LLM opportunity

LLMs are trained on financial disclosures, analyst reports, and economic news. They can incorporate qualitative dimensions that time-series models miss. The empirical question is whether this yields a measurable improvement in forecast accuracy.

Chain-of-Thought Cash Flow Forecasting

Wei et al. (2022): CoT prompting elicits intermediate reasoning steps before the final answer. For forecasting, CoT forces the model to articulate causal mechanisms — making output auditable.

Four-stage CoT forecasting prompt:

1. **Macro context**: how do interest rates, GDP, and inflation affect revenue and margins?
2. **Industry dynamics**: drivers and headwinds; competitive dynamics; regulatory changes.
3. **Firm-specific**: competitive position; margin trajectory; CapEx plans from MD&A.
4. **Synthesis**: point estimate + upside (P90) + downside (P10) per year; 2-sentence justification.

Why separating reasoning from numbers matters

If the reasoning says "highly competitive market with stable pricing" but the number says 20% revenue growth, an automated consistency-checker

Evaluating Forecast Quality

Mean Absolute Percentage Error (MAPE):

$$\text{MAPE} = \frac{1}{n}\sum_{t=1}^{n}\frac{|y_t - \hat{y}_t|}{|y_t|} \times 100$$

Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}$$

Directional Accuracy (DA):

$$\text{DA} = \frac{1}{n}\sum_{t=1}^{n}\mathbf{1}[\mathrm{sign}(y_t - y_{t-1}) = \mathrm{sign}(\hat{y}_t - y_{t-1})]$$

Metric	Problem	When to use
MAPE	Undefined when $y_t \approx 0$	Stable, positive cash flows
RMSE	Scale-dependent	Within-company comparison
DA	Scale-invariant	Investment decision relevance

Why LLMs Cannot Be Trusted for Arithmetic

Architectural mismatch

LLMs are next-token predictors. Arithmetic is a deterministic algorithm. Statistical pattern-matching approximates arithmetic for simple cases; reliability degrades rapidly with complexity.

For business valuation this is serious:

- A 5-year DCF involves $\approx$ 15–20 chained arithmetic operations
- An incorrectly applied tax rate propagates through every calculation
- Compounding errors: a 5% FCFF extraction error + 1% terminal growth bias can together cause a $>$ 10% error in enterprise value

Solution: architectural, not prompting. Remove arithmetic from the LLM entirely. Route all computation to a deterministic executor.

Rule

No financial calculation — however simple — should be performed by in-context LLM reasoning.

Code Interpreter Pattern for DCF (1/2)

```
# ReAct agent trace --- DCF computation

Thought 1: I need to compute FCFF. EBIT=840, DA=310, CapEx=180,
           delta_NWC=80, tau=0.21. Use python_executor.

Action 1:  python_executor(code="""
               ebit=840; da=310; capex=180; delta_nwc=80; tau=0.21
               fcff = ebit*(1-tau) + da - capex - delta_nwc
               print(f"FCFF = {fcff:.1f} M")
           """)

Observation 1: FCFF = 713.6 M
```

The LLM emits structured Thought/Action steps; arithmetic is delegated to `python_executor` rather than performed in-context.

Code Interpreter Pattern for DCF (2/2)

```
Thought 2: Now compute the 5-year DCF with g=8%, g_term=3%, WACC
    =9%.

Action 2:  python_executor(code="""
               import numpy as np
               fcff0=713.6; g=0.08; g_t=0.03; wacc=0.09; n=5
               fcfs=[fcff0*(1+g)**t for t in range(1,n+1)]
               tv=fcfs[-1]*(1+g_t)/(wacc-g_t)
               ev=sum(f/(1+wacc)**t for t,f in enumerate(fcfs,1))+
                   tv/(1+wacc)**n
               print(f"EV = {ev:.1f} M")
           """)

Observation 2: EV = 15224.0 M
```

Generated code is an auditable artefact: human reviewers inspect it to verify formula, inputs, units.

Four LLM Failure Modes in Valuation

1. **Phantom tickers**: the LLM generates realistic-sounding but non-existent tickers. *Mitigation*: validate every ticker against a securities master before use.
2. **Invented ratios**: confabulated plausible-looking numbers not in any filing. *Mitigation*: trace every figure to a specific EDGAR accession number. Replace "trust but verify" with **"do not trust until verified."**
3. **Misapplied formulas**: syntactically correct code implementing the wrong formula (FCFE when FCFF was requested; missing tax shield on interest). *Mitigation*: automated formula-level testing against known benchmark cases.
4. **Stale knowledge**: LLMs have a knowledge cutoff. Financial data, tax rates, and accounting standards change. *Mitigation*: always use EDGAR-retrieved information in preference to parametric knowledge for specific company facts.

Scenario Construction: Bull, Base, and Bear

Scenario	Percentile	Description
Bull	P90	Favourable macro, strong execution, above-consens
Base	P50	Median expectation; most likely trajectory
Bear	P10	Adverse macro, margin compression, or competitiv

Internal consistency requirement: every scenario must specify a complete, internally consistent set: revenue growth, EBIT margins, CapEx intensity, NWC ratios, and terminal growth rate. A scenario combining 25% revenue CAGR and 1% terminal growth is internally contradictory.

LLM role in scenarios

LLMs excel at *narrative coherence checking*: given a proposed set of scenario inputs, does the narrative justify the numbers? Do the numbers contradict the stated macro environment? **Not** at computing the resulting valuations — that goes to the code interpreter.

Monte Carlo DCF: Full Distribution of Enterprise Value

Let WACC $\sim \mathcal{N}(\mu_W, \sigma_W^2)$ and $g \sim \mathcal{N}(\mu_g, \sigma_g^2)$ (truncated so $g <$ WACC). For M simulation paths:

$$V^{(m)} = \sum_{t=1}^{n} \frac{\mathrm{FCF}_t^{(m)}}{(1+\mathrm{WACC}^{(m)})^t} + \frac{\mathrm{FCF}_{n+1}^{(m)}}{(\mathrm{WACC}^{(m)} - g^{(m)})(1+\mathrm{WACC}^{(m)})^n}$$

Typical parameterisation: $\mu_W = 0.09$, $\sigma_W = 0.015$, $\mu_g = 0.03$, $\sigma_g = 0.01$. With $M = 10{,}000$: runtime < 1 second.

Output: mean, median, standard deviation, P10, P25, P75, P90 of EV distribution.

Why Monte Carlo is not optional

A 100-bps error in terminal growth $\rightarrow$ **15–18% error in enterprise value** (see error propagation analysis). Point estimates are not just imprecise — they are misleading.

Comparable Selection: Pitfalls and LLM Solutions

Traditional pitfalls:

1. **Code obsolescence**: SIC codes last revised in 1987. A cloud computing company and a 1990s software distributor share the same SIC.
2. **Diversified conglomerates**: manufacturing SIC company deriving revenue from financial services.
3. **Cyclical timing**: temporarily distressed firms depress the median multiple.

Two LLM-based approaches:

Prompt-based: describe the target in natural language; request 8–12 comparable firms as JSON. *Problem*: hallucinated tickers occur frequently. *Mandatory*: validate every ticker against a securities master.

Comparable Selection: Embedding-Based Approach

UNDER THE HOOD

Embedding-based: embed the MD&A of all companies in the universe; retrieve top-k by cosine similarity.

$$\mathrm{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

Empirical result: embedding-based peers have lower IQR of EV/EBITDA and lower deviation from transaction prices vs. SIC-based peers.

Trade-off: embeddings are grounded (every retrieved peer is a real firm) but may surface textual lookalikes whose business model differs from the target. The next slide combines both approaches to capture their complementary strengths.

Hybrid Comparable Pipeline

Neither approach alone is sufficient:

- Prompt-based: broad world knowledge but hallucinates tickers
- Embedding-based: grounded but may surface textual lookalikes with different businesses

Two-stage hybrid:

1. **Candidate generation**: embedding index returns top 30 by cosine similarity.
2. **LLM filtering**: model receives candidates + brief company descriptions; removes firms that are geographically incomparable, in different competitive positions, or subject to idiosyncratic events (pending acquisition, bankruptcy). Filtered set of 8–12 comparables proceeds to multiple analysis.

Principle: confine the LLM to its strengths

Stage 1: grounded retrieval (no hallucination risk — candidates come from real company vectors). Stage 2: qualitative business-model reasoning (LLM excels). Stage 2 *never* needs to recall whether a specific ticker exists — that is already guaranteed by Stage 1.

Error Propagation in Multi-Step Pipelines

Suppose extraction produces FCF with error ε_1 and forecasting introduces a terminal growth bias ε_2. First-order approximation:

$$\Delta V \approx \frac{\partial V}{\partial \mathrm{FCF}}\varepsilon_1 + \frac{\partial V}{\partial g}\varepsilon_2$$

For the Gordon Growth perpetuity:

$$\frac{\partial V}{\partial \mathrm{FCF}_n} = \frac{1+g}{\mathrm{WACC}-g}, \qquad \frac{\partial V}{\partial g} = \frac{\mathrm{FCF}_n(1+\mathrm{WACC})}{(\mathrm{WACC}-g)^2}$$

At WACC = 9%, g = 3%:

- 1% relative error in FCF $\rightarrow \approx$ 1% error in enterprise value
- **100-bps error in terminal growth $\rightarrow \approx$ 15–18% error in enterprise value**

The dominant uncertainty source

Terminal growth rate uncertainty *dominates* enterprise value uncertainty. Always run Monte Carlo simulation; always present percentile ranges.

Best Practices for Production Deployment

1. **XBRL first**: verify extracted numbers against XBRL tags. Flag any conflict for human review. LLM extraction is a complement, not a sole source.
2. **Code interpreter for all arithmetic**: no financial calculation should be performed by in-context LLM reasoning. Generated code is the deliverable.
3. **Run Monte Carlo**: a point-estimate DCF is misleading. Communicate the P10–P90 range alongside the central estimate.
4. **Mandatory hallucination guards**: validate every ticker against a securities register; trace every financial figure to a specific EDGAR accession number; suppress unverifiable claims from final output.
5. **Human review before publication**: LLMs are a powerful analytical assistant, not an autonomous decision-maker. A qualified analyst must take responsibility for the figures.

Consistent with FSB (2023) guidance on AI in finance: human accountability, model transparency, and robust back-testing are prerequisites.

Summary: The LLM-Assisted Valuation Stack

Component	Key mechanism
Data extraction	EDGAR API + LLM table parsing + XBRL validatio
FCF computation	Pydantic schema + code interpreter
Forecasting	CoT prompting; evaluate with MAPE / RMSE / DA
Scenarios	Bull/base/bear + Monte Carlo over WACC and g
Comparable selection	Embedding index + LLM qualitative filter
Orchestration	Multi-agent with parallel specialist execution
Governance	HITL review; hallucination guards; audit trail

Connection to Chapter 6

Chapter 6 applies the same data extraction and forecasting tools to credit risk — where target variables are probability of default and loss given default, and regulatory constraints on model transparency are stricter.

Appendix

Worked Case Study and Production Orchestration

Extra / optional material — beyond the core lecture.

Case Study: Mid-Cap SaaS Company

Profile: Revenue \$4.2B (+12% YoY), EBIT margin 20%, D&A \$310M, CapEx \$180M, net debt \$800M, 280M shares outstanding.

After normalisation: EBIT = \$915M, FCFF = \$789M (two non-recurring items: \$45M restructuring + \$30M acquired intangible amortisation).

Method	Enterprise Value	Equity/share
DCF (WACC 9%, g 3%)	\$17.0B	\$57.9
Comparables (EV/EBITDA 24.5×)	\$29.9B	—

DCF vs. comparables gap: sector trading at elevated growth multiples. Both are informative; neither is definitive.

Monte Carlo (10,000 draws): mean \$17.2B, P10–P90 range \$12.5B–\$24.1B. The range quantifies the uncertainty a point estimate conceals.

Multi-Agent Orchestration

Why not a monolithic script? A failure in step 6 (terminal value) requires rerunning all prior steps. A monolithic pipeline is also untestable in parts.

Orchestrator + specialists:

- `edgar_agent`: retrieval and extraction
- `fcf_agent`: computation and normalisation
- `forecast_agent`: CoT projections
- `comps_agent`: comparable selection
- `dcf_agent`: final valuation and scenario synthesis

Parallelism: comparable selection has no dependency on the FCF forecast — both can run concurrently, reducing end-to-end latency.

Production latency

6 LLM calls $\times$ 1–5 seconds each + parallelisation $\approx$ **15–30 seconds end-to-end** per company valuation. Cost: \$0.50–\$3.00 per company at 2024

FCFF vs. FCFE: Discount-Rate Mapping

For reference, the two free-cash-flow definitions are discounted at different rates and yield different value measures:

Metric	Discounted at	Yields
FCFF	WACC	Enterprise value
FCFE	Cost of equity	Equity value directly

Discounting FCFF at WACC gives enterprise value; subtract net debt to recover equity value. Discounting FCFE at the cost of equity yields equity value directly.